

Angular v17

Built-in control flow

① Conditional Statement



```
<div *ngIf="loggedIn; else anonymousUser">
```

The user is logged in

```
</div>
```

```
<ng-template #anonymousUser>
```

This user is not logged in

```
</ng-template>
```



```
@if (loggedIn) {
```

The user is logged in

```
} @else {
```

The user is not logged in

```
}
```



```
<div [ngSwitch]="accessLevel">
```

```
  <admin-dashboard *ngSwitchCase="admin"/>
```

```
  <moderator-dashboard *ngSwitchCase="moderator"/>
```

```
  <user-dashboard *ngSwitchDefault />
```

```
</div>
```



```
@switch (accessLevel) {
```

```
  @case ('admin') { <admin-dashboard /> }
```

```
  @case ('moderator') { <moderator-dashboard /> }
```

```
  @default { <user-dashboard /> }
```



```
@pr (user of users; track user.id) {
```

```
  {{ user.name }}
```

```
} @empty {
```

```
  Empty list of users
```

```
}
```

Deferrible views

Deferrible views bring performance and developer experience to the next level because they enable declarative & powerful deferred loading with unprecedented ergonomics.

Currently you have to use `viewContainerRef` while also managing all the complexity for cleanups, managing loading error, showing a placeholder etc. Taking care of various corner cases may result in some non-trivial code, which will be hard to test & debug.

The new deferrible view, allow you to lazily load the list of comments and all their transitive dependencies with a single line of declarative code.

```
@defer {  
  <comment-list>  
}
```

Starting to lazily load a component when a certain DOM element enters the viewport involves a lot of more non-trivial logic and the IntersectionObserver API. Angular makes using IntersectionObservers as simple as adding a deferrable view trigger!

```
@defer (on viewport) {  
  <comment-list>  
}  
@placeholder {  
  <img src = "comments-placeholder.png">  
}
```

```
@defer (on viewport) {
```

```
  <comment-list>
```

```
} @loading {
```

```
  Loading
```

```
} @error {
```

```
  Loading failed
```

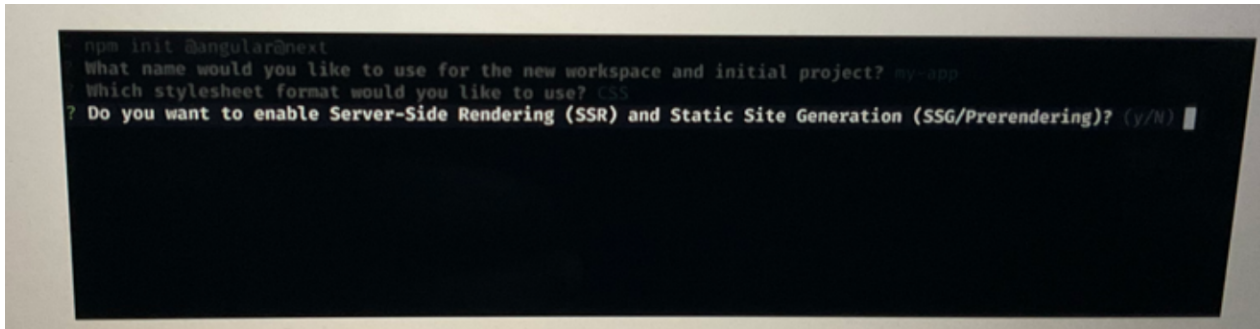
```
} @placeholder {
```

```
  
```

```
}
```

Revamped hybrid rendering experience

SSR and SSG closer to developers with a prompt in `ng new`.



```
ng new my-app --ssr
```

```
ng add @angular/ssr
```

New lifecycle hooks

→ `afterRender` - register a callback to be invoked each time the application finishes rendering.

→ `afterNextRender` - register a callback to be invoked the next time the application finishes rendering.

```
@Component({
  selector: 'my-chart-cmp',
  template: '<div #chart>{{.....}}</div>'
})
```

```
export class MyChartCmp {
  @ViewChild('chart') chartRef: ElementRef;
  chart: MyChart | null;
```

```
  constructor() {
```

```
    afterNextRender(() => {
```

```
      this.chart = new MyChart(this.chartRef.  
                                nativeElement);  
    }, { phase: AfterRenderPhase.work });
```

```
  }  
}
```


Input value transform

```
@component({  
  standalone: true,  
  selector: 'my-expander',  
  template: '...'  
})
```

```
export class Expander {  
  @Input expanded: boolean = false;  
}
```

```
<my-expander expanded />
```

```
@component({  
  standalone: true,  
  selector: 'my-expander',  
  template: '...'  
})
```

```
export class Expander {  
  @Input({ transform: booleanAttribute })  
    expanded: boolean = false;
```


Style and Stylesheets as String



```
@Component({  
  styles: ['.....']  
})
```

```
@Component({  
  styleUrls: ['styles.css']  
})
```



```
@Component({  
  styles: '.....'  
})
```

```
@Component({  
  styleUrls: 'styles.css'  
})
```